Android with Arduino

There must be a lot of people out there who love trying out things with Android, and who spend a lot of time experimenting with it. Here's something for those who love creating things with FOSS and electronic prototyping boards.

Arduino is an open source, single microcontroller electronics prototyping board with easy-to-use hardware and software. It was developed in 2005 by Massimo Banzi and David Cuartielles. Arduino is capable of interacting with the environment by receiving inputs from a broad range of sensors and responding by sending outputs to various actuators. The Arduino board consists of 8-bit Atmel AVR microcontrollers, in addition to which, the boards have a standard way of connecting the CPU with various other complementary components to increase its functionality through a number of add-ons called shields. An Arduino board can be built by hand or can be purchased (pre-assembled) from *http://arduino.cc/en/Main/Buy*, depending on your needs.

Installing and working with Arduino

The microcontroller on the Arduino boards is programmed via the Arduino Programming Language (based on Wiring) and Arduino Development Environment (based on Processing). The open source Arduino environment can be downloaded for Windows, Linux or Mac OS X from *http://arduino.cc/en/Main/Software* and extracted. This makes writing code and uploading it to the board very easy. As the environment is written in Java, make sure you have Java installed.

When you are finished with the installation, here's a program to start off with Arduino programming. All programmers must be familiar with the 'Hello World' sample. When working with electronic prototyping boards, i.e., during physical computing—for micro-controllers that don't have a display device, an LED is added. So just start the Arduino software, select your board model and enter the following code:

```
int ledPin = 13;                  // LED connected to digital pin
13

void setup() {
  pinMode(ledPin, OUTPUT);      // sets the digital pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH);   // sets the LED on
  delay(1000);                  // waits for a second
  digitalWrite(ledPin, LOW);    // sets the LED off
  delay(1000);                  // waits for a second
}
```

Once you've typed in the code, connect your board via USB, and upload the program to it. As the LED has polarity, you need to fix it onto the board carefully. The long leg, typically positive, should be connected to pin 13, and the short leg to GND (i.e., ground). The LED starts going ON and OFF at intervals of one second, as shown in Figure 1.

Connecting Arduino to Android

To connect your Arduino board to an Android device, you need to have an 'Amarino toolkit'. Amarino is a project developed at MIT to connect the Arduino and Android via